

Question Bank — Week 15

The Principles of Language Translation (Lecture 15) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice	2
Part B — Short answer and essay	5
Part C — Trace the code	5
Part D — Trees and stacks: draw every step	6
Part E — Complexity analysis	6
Part F — Find and fix the bug	6
Part G — Write the code	7

Answers are in a separate file: [week15-answers.md](#). Levels: **[what]** recall · **[how]** apply · **[why]** explain and justify. The functions are those of `dsa/translation.py`: `tokenize`, `Parser` (with `parse`, `expr`, `term`, `factor`, `peek` and `advance`), `evaluate` and `calculate`, with the tree nodes `Num(value)` and `BinOp(op, left, right)`. *Unary minus `-x` is parsed as `BinOp("-", Num(0.0), x)`. `infix_to_postfix` and `evaluate_postfix` are Week 6's. Unless a question says otherwise, the grammar is*

```
expr  := term (('+' | '-') term)*
term  := factor (('*' | '/') factor)*
factor := NUMBER | '(' expr ')' | '-' factor
```

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W15-M01 – W15-M22
B	Short answer and essay	W15-E1 – W15-E5
C	Trace the code	W15-T1 – W15-T5
D	Trees and stacks — draw every step	W15-S1 – W15-S3
E	Complexity analysis	W15-K1 – W15-K3
F	Find and fix the bug	W15-B1 – W15-B4
G	Write the code — checked by pytest	W15-C1 – W15-C5

Part A — Multiple choice

W15-M01 [what] The job of the **lexer** is to:

- **a)** compute the value of the expression
- **b)** group the characters of the text into tokens
- **c)** check that the parentheses are balanced
- **d)** build the syntax tree

W15-M02 [how] `tokenize("12+3")` returns a list of how many tokens?

- **a)** 4
- **b)** 2
- **c)** 3
- **d)** 5

W15-M03 [what] `calculate("3 + * 4")` fails. Which stage rejects it?

- **a)** the parser
- **b)** the lexer
- **c)** the evaluator
- **d)** none — it returns 7

W15-M04 [what] “AST” stands for:

- **a)** array of sorted tokens
- **b)** automatic syntax translator
- **c)** algebraic stack tree
- **d)** abstract syntax tree

W15-M05 [why] In the grammar, `*` binds more tightly than `+` because:

- **a)** the parser keeps a table in which `*` has the higher number
- **b)** `*` comes before `+` in ASCII
- **c)** `*` is handled in `term`, a rule that `expr` is built from
- **d)** `*` is always evaluated from right to left

W15-M06 [how] `calculate("2 + 3 * 4 - 1")` returns:

- **a)** 19.0
- **b)** 13.0
- **c)** 11.0
- **d)** 15.0

W15-M07 [how] The **root** of the AST of `2 * 3 + 4` is:

- **a)** +
- **b)** *
- **c)** 2
- **d)** 4

W15-M08 [how] The root of the AST of `(1 + 2) * (3 - 4)` is:

- **a)** +
- **b)** -
- **c)** (
- **d)** *

W15-M09 [why] $1 - 2 - 3$ evaluates to -4 because subtraction is:

- **a)** commutative
- **b)** left-associative: $(1 - 2) - 3$
- **c)** right-associative: $1 - (2 - 3)$
- **d)** of higher precedence than itself

W15-M10 [why] A student writes `expr` right-recursively: parse a term; **if** the next token is + or -, return `BinOp(op, term, self.expr())`. What does it give for $8 - 4 - 2$?

- **a)** 2
- **b)** an error
- **c)** 6
- **d)** 14

W15-M11 [why] The rule `expr := expr '+' term | term` is turned directly into a method whose first line is `left = self.expr()`. On the input 3 it:

- **a)** raises `RecursionError` — it recurses without consuming a token
- **b)** returns `Num(3.0)`
- **c)** raises `ValueError: empty expression`
- **d)** loops for ever without using any memory

W15-M12 [what] In recursive descent, the EBNF repetition `( ... )*` becomes:

- **a)** a recursive call
- **b)** a stack
- **c)** an if
- **d)** a while loop

W15-M13 [how] Parsing $1 + 2 * 3$, how many times is `factor` called?

- **a)** 1
- **b)** 2
- **c)** 3
- **d)** 5

W15-M14 [how] The tree our parser builds for $-5 + 2$ is:

- **a)** `BinOp("-", Num(0.0), BinOp("+", Num(5.0), Num(2.0)))`
- **b)** `BinOp("+", BinOp("-", Num(0.0), Num(5.0)), Num(2.0))`
- **c)** `BinOp("+", Num(-5.0), Num(2.0))`
- **d)** `BinOp("-", Num(5.0), Num(2.0))`

W15-M15 [what] `evaluate` visits the nodes of the tree in:

- **a)** post-order — both children before the node
- **b)** pre-order

- **c)** in-order
- **d)** level-order

W15-M16 [how] The post-order sequence of the AST of $(3 + 4) * 2$ is:

- **a)** $* + 3 4 2$
- **b)** $3 + 4 * 2$
- **c)** $2 3 4 + *$
- **d)** $3 4 + 2 *$

W15-M17 [why] `parse` raises `ValueError` if tokens are left over after `expr` returns. Without that check:

- **a)** `"3 +"` would return 3
- **b)** `"3 + 4 )"` would return 7
- **c)** `"(3 + 4"` would return 7
- **d)** nothing would change

W15-M18 [how] Each pair of parentheses costs the parser about three Python frames. With Python's default limit of 1,000 frames, roughly how deeply can calculate nest parentheses?

- **a)** 1,000
- **b)** 100
- **c)** 330
- **d)** without limit

W15-M19 [how] The time of calculate on an expression of n tokens is:

- **a)** $O(n)$
- **b)** $O(n \log n)$
- **c)** $O(n^2)$
- **d)** $O(\log n)$

W15-M20 [why] `calculate("1 + 1 + ... + 1")` with n numbers: how deep does the recursion of `evaluate` go?

- **a)** 1 — the parser used a loop
- **b)** about $\log_2 n$
- **c)** 3 per number
- **d)** about n — the tree leans left all the way down

W15-M21 [what] When you run a `.py` file, CPython:

- **a)** interprets the text line by line, with no compilation
- **b)** compiles it to machine code, like a C compiler
- **c)** compiles it to bytecode, then interprets the bytecode
- **d)** translates it to Java

W15-M22 [why] Compared with the Week 6 route (`shunting-yard`, then `evaluate_postfix`), the main advantage of building a tree is that:

- **a)** it is asymptotically faster

- **b)** the tree can be reused — printed, simplified, compiled — and new grammar rules such as unary minus are easy to add
 - **c)** it needs no recursion
 - **d)** it needs no memory beyond the tokens
-

Part B — Short answer and essay

W15-E1 [what] (4 marks) Name the three stages of a translator, with the input and output of each. For each, give an input that it — and only it — rejects, and the exception our calculator raises. How does a compiler differ from an interpreter?

W15-E2 [why] (4 marks) “Precedence is the shape of the grammar.” Explain this sentence using the three rules of our grammar and the tree of $3 + 4 * 2$. How do parentheses override precedence? Where would you add a rule for a tighter operator such as $^$?

W15-E3 [why] (4 marks) What is associativity? Explain why `expr` must use a **loop** that makes the tree so far the **left** child. Describe what goes wrong with a right-recursive `expr` and with a left-recursive one, with an input for each.

W15-E4 [why] (3 marks) Compare the two routes from infix tokens to a value: shunting-yard plus `evaluate_postfix`, and `Parser` plus `evaluate`. Give their costs, one advantage of each, and the connection between the postfix list and the tree.

W15-E5 [why] (3 marks) “In a recursive-descent parser, the call stack is the parser’s stack.” Explain. What decides how deep it grows, and what happens when it grows too deep?

Part C — Trace the code

W15-T1 [how] Give `tokenize(" 4.5*(10-3) ")`. For each token, give the index in the text at which it starts, and say what the lexer does with the spaces.

W15-T2 [how] Trace `Parser(tokenize("2 * (3 - 1) + 4")).parse()` as an indented list of calls of `expr`, `term` and `factor`, one line per call, showing the token each call starts at. Give the tree and its value.

W15-T3 [how] For $(8 - 2) / (1 + 2) * 3$, draw the tree, then list the order in which `evaluate` **finishes** its nodes, with each node’s value. What is the result?

W15-T4 [how] Evaluate $2 * (3 - 1) + 4$ by the **stack** route: `infix_to_postfix`, then `evaluate_postfix`. Give the postfix list and the value stack after each token. Compare with the tree of W15-T2.

W15-T5 [how] What does `calculate` return, or raise (the exception type and the stage), for each of: `"2 * -3"`, `"--4"`, `"4 -"`, `"(2 + 3) (4)"`, `"6 / (3 - 3)"`, `" "`, `"2 ^ 3"`, `"(2)"`?

Part D — Trees and stacks: draw every step

W15-S1 [how] Draw the AST our parser builds for each expression, and give its value: (a) $4 + 2 * 3$; (b) $(4 + 2) * 3$; (c) $9 - 3 - 2$; (d) $-3 * -2$; (e) $12 / 2 / 3 * 4$. Then draw the tree a **right-recursive** expr would build for (c), and its value.

W15-S2 [how] expr parses $10 - 2 + 3 - 1$. Draw the tree held in node after node = self.term() and after **each** turn of the while loop. What is the final value?

W15-S3 [how] $(1 + 2) * (3 - 4)$: give its postfix list, and draw the value stack of evaluate_postfix after each token. Then draw the AST and number its nodes in post-order. What do you notice?

Part E — Complexity analysis

W15-K1 [how] Our tokenize scans with an index i and cuts each number out with one slice. A student instead writes the loop as while text: ch = text[0]; ...; text = text[1:]. Give Θ of both versions for a text of n characters, and justify.

W15-K2 [why] Give the depth of the Python call stack — as a function of k — reached by (a) the **parser** and (b) evaluate, for three inputs: k nested pairs of parentheses around 1; the flat chain $1 + 1 + \dots + 1$ with k numbers; and a balanced, fully parenthesised expression with k numbers. Which of these can raise RecursionError for k = 1,000, and in which function?

W15-K3 [how] Two ways to write to_postfix(node) (W15-C1):

```
def concat(node):
    if isinstance(node, Num):
        return [number_text(node.value)]
    return concat(node.left) + concat(node.right) + [node.op]
```

and a version that appends to one shared output list during a post-order walk. Give Θ of each for a tree of n nodes, in the worst case. Which tree shape is the worst case for the first?

Part F — Find and fix the bug

W15-B1 [why]

```
def expr(self):
    node = self.term()
```

```
if self.peak() in ("+", "-"):
    op = self.advance()
    return BinOp(op, node, self.expr())
return node
```

W15-B2 [how]

```
def factor(self):
    token = self.peak()
    if token is None:
        raise ValueError("unexpected end of input")
    if token == "(":
        self.advance()
        node = self.expr()
        if self.peak() != ")":
            raise ValueError("missing ')")
        return node
    if token == "-":
        self.advance()
        return BinOp("-", Num(0.0), self.factor())
    if token[0] in "0123456789.":
        self.advance()
        return Num(float(token))
    raise ValueError(f"unexpected {token!r}")
```

W15-B3 [why] Part of factor:

```
if token == "-":
    self.advance()
    return BinOp("-", Num(0.0), self.expr())
```

W15-B4 [how] Part of tokenize:

```
elif ch in DIGITS or ch == ".":
    start = i
    while text[i] in DIGITS or text[i] == ".":
        i += 1
    tokens.append(text[start:i])
```

Part G — Write the code

In `practice/week15.py`; check with `pytest tests/test_practice_week15.py -v`. The trees are the `Num` and `BinOp` of `dsa/translation.py`. Do not use `eval`.

W15-C1 [how] `to_postfix(node)` — the postfix token list of a tree, by a post-order walk. Why is it the same list as `infix_to_postfix` gives?

W15-C2 [how] `to_infix(node)` — fully parenthesised infix: `"(3 + (4 * 2))"`.

W15-C3 [why] `to_infix_minimal(node)` — infix with only the parentheses the grammar needs, so that the result parses back to the **same** tree. When does a child need parentheses?

W15-C4 [how] `postfix_to_tree(tokens)` — build the tree from a postfix list with a stack of subtrees; `ValueError` on malformed input.

W15-C5 [why] `calculate_with_power(tokens)` — add a right-associative `^` that binds tighter than unary minus on its left, as `**` does in Python: `2 ^ 3 ^ 2` is 512 and `-2 ^ 2` is -4. Why does `^` use recursion where `+` uses a loop?